CS 422 Assignment 3: TCP Congestion Control Implementation (Option 1: eBPF)

Thanh Bui Minh (tbuiminh) Aaron Gupta (gupt1066) Rishiraj Behki (rbehki)
Pranith Ravella (pravella) Colin Hermack (chermack)

Fall 2026

Repository: https://github.com/pkravella/cs422-project3

We implemented the algorithm from our Assignment 2, Question 3(b) design, **Delay-Gated Loss (DGL)**, as an eBPF `struct_ops` program registered with the kernel as `cs422cc`, and compared it with CUBIC, Reno, and BBR on 10 public iperf3 servers.

1. Algorithm implementation

All of the algorithm is in `cc/cs422cc.bpf.c`. It implements five hooks of the kernel's `struct tcp_congestion_ops`, registered in the `cs422cc_ops` struct. Each hook is a BPF program that the kernel calls on that socket's ACK and loss events:

Hook	Kernel calls it	What DGL does there
`init`	once, when the handshake completes	reset the per-connection state
`pkts_acked`	every ACK of new data, also during loss recovery, before `cong_avoid`	collect the three signals
`cong_avoid`	every ACK outside loss recovery	grow, hold, or back off cwnd
`ssthresh`	on loss (fast retransmit or timeout)	classify the loss, return the reduced window
`undo_cwnd`	when a loss turns out to be spurious	restore the window (as Reno does)

The three signals

All three are collected in `pkts_acked`, because it runs on every ACK, including during recovery, when `cong_avoid` is not called.

- **RTT.** Each ACK's RTT sample feeds `update_min_rtt()`: `min_rtt` is the smallest sample in the last 10 s. The queueing delay is `srtt - min_rtt`, using the kernel's smoothed RTT (`tcp_sock.srtt_us`, which the kernel stores multiplied by 8). `queue_longer_than()` compares it with `min_rtt / 8` ("is any queue forming?") or `min_rtt / 4` ("is the queue long?").
- **Throughput.** A *round* is one round trip: it ends once the data that was in flight when it started has been delivered (`start_round()`). At the end of each round, `finish_round()` computes the round's delivery rate (segments delivered / round duration) and keeps `bw_max = max(rate, bw_max * 15/16)`.
- **Loss proxy.** The same function records the segments retransmitted and delivered in the round (from `tcp_sock.total_retrans` and `tcp_sock.delivered`). `last_round_loss_heavy()` reports whether more than 5% of the last round was retransmitted.

How the window is updated

On each ACK (`cong_avoid`), only if the sender is actually using its window (`is_cwnd_limited`, mirroring the kernel's `tcp_is_cwnd_limited()`):

- *Slow start.* If a queue has started forming (`queue_delay > min_rtt / 8`), set `ssthresh = cwnd`, which ends slow start. Otherwise grow by one segment per segment acknowledged, which doubles cwnd every RTT.
- *No queue* (`queue_delay <= min_rtt / 8`): additive increase, one segment per RTT (`additive_increase()`).
- *Long queue* (`queue_delay > min_rtt / 4`): cut cwnd by 1/8 to drain the queue, at most once per round (`delay_backoff()`).
- *Small queue* (in between): hold cwnd steady.

On loss (`ssthresh`), the loss is classified:

- *Congestion* (long queue, or the 5% loss-rate guard): return `cwnd / 2`, Reno's cut.
- *Random* (no long queue): return `cwnd * 7/8`, but not less than the measured bandwidth-delay product `bw_max * min_rtt` (`bdp_floor_segments()`).

After fast recovery the kernel sets cwnd to the returned ssthresh; after a timeout it sets cwnd to 1 and slow-starts back up to it.

The kernel has no floating point, so every fraction is an integer numerator and denominator, and the bandwidth and BDP arithmetic is done in 64-bit integers so that `segments * 10^6` cannot overflow.

Initial state and parameters

Per-connection state is the `struct dgl_state`, stored in the socket's `icsk_ca_priv` scratch area like the built-in algorithms' state. A `_Static_assert` checks at compile time that it fits, using the area's size from the kernel's own type information.

Initial state (`init`): no `min_rtt` yet (a sentinel value meaning "unset"; until the first sample DGL reports no queue and behaves like Reno), `bw_max = 0`, no completed round, and a first round starting at the handshake. cwnd and ssthresh keep the kernel's defaults: **cwnd = 10 segments, ssthresh = "infinite"** (`TCP_INFINITE_SSTHRESH`), so every connection starts in slow start.

Parameters (constants), unchanged from Assignment 2:

Constant	Value	Meaning
`GROW_DIVISOR`	8	grow only while queueing delay <= min_rtt / 8; also ends slow start
`CONGESTED_DIVISOR`	4	queueing delay > min_rtt / 4 means congestion
`CONGESTION_BETA`	1/2	window after a congestion loss
`RANDOM_LOSS_BETA`	7/8	window after a random loss
`DELAY_BACKOFF_DIVISOR`	8	cut cwnd / 8 per round while the queue is long
`LOSS_RATE_LIMIT`	5/100	per-round retransmission rate treated as congestion
`BW_DECAY`	15/16	per-round decay of `bw_max`
`MIN_RTT_WINDOW_US`	10 s	forget `min_rtt` after this long without a new minimum
`MIN_CWND`	2 segments	smallest window ever set

Units: windows and counters are in segments, times in microseconds, and rates in segments per second.

Changes from the Assignment 2 pseudocode

1. **The delay backoff also lowers ssthresh.** The pseudocode only cut cwnd. That leaves cwnd below ssthresh, so on the next ACK the kernel would treat the connection as in slow start and double the window we just cut. `delay_backoff()` sets `ssthresh = cwnd` after the cut.
2. **The random-loss floor is capped at the current cwnd.** `bw_max` can come from an earlier, larger window, so `bw_max * min_rtt` can exceed the current window. A loss must never raise the window, so `ssthresh` caps the result at cwnd.
3. **Implementation details the pseudocode left open:** the signals are collected in `pkts_acked` rather than in the window-update hook, because `cong_avoid` is not called during loss recovery; round boundaries use the kernel's `tcp_packets_in_flight()` formula and a wraparound-safe counter comparison; and the "no queue" test is `<= min_rtt / 8` rather than `<`, which makes no practical difference.

Everything else (thresholds, cut sizes, loss classification, the BDP floor, and the loss-rate guard) is as in the pseudocode.

2. Kernel, build, and load

Measurement host: Ubuntu 24.04.4 LTS KVM virtual machine (a VPS), x86_64, kernel `6.8.0-117-generic`, default congestion control `cubic`. The same program also builds, passes the verifier, and registers on the WSL2 kernel `6.18.33.2-microsoft-standard-WSL2` (clang 21).

Required kernel features: BPF `struct_ops` for `tcp_congestion_ops` (Linux 5.6+), with `CONFIG_BPF_SYSCALL=y`, `CONFIG_BPF_JIT=y`, and `CONFIG_DEBUG_INFO_BTF=y` (the kernel's type information, exposed as `/sys/kernel/btf/vmlinux`). BBR is `CONFIG_TCP_CONG_BBR=m`, a module loaded with `modprobe tcp_bbr`. All four are present on the measurement host.

Toolchain on the measurement host: clang 18.1.3, bpftool 7.4.0 (libbpf 1.4), `libbpf-dev` 1.3.0, Python 3.12.3.

Build and load (from the repository root):

```
sudo apt install -y build-essential clang llvm libbpf-dev \
    linux-tools-common linux-tools-$(uname -r)
cc/load.sh      # build, register cs422cc, load bbr, allow both
cc/unload.sh    # unregister cs422cc
```

`cc/Makefile` first dumps `vmlinux.h` (C definitions of every kernel struct) from the running kernel's BTF, then compiles with `clang -target bpf -O2 -g`. It builds with no warnings under `-Wall`. `cc/load.sh` runs `bpftool struct_ops register cc/cs422cc.bpf.o`, which loads the five programs through the BPF verifier and registers `cs422cc_ops` as a new algorithm. It then loads `tcp_bbr` and adds `cs422cc` and the baselines to `net.ipv4.tcp_allowed_congestion_control`, so an unprivileged socket may select them. On the measurement host the output was:

```
Registered tcp_congestion_ops cs422cc_ops id 96
available: reno cubic cs422cc bbr
allowed:   reno cubic cs422cc bbr
```

Selecting it on the data socket. The Assignment 2 client calls `setsockopt(IPPROTO_TCP, TCP_CONGESTION, "cs422cc")` on the data socket before `connect()`, then reads the algorithm back with `getsockopt()` (`open_data_stream()`). That value is saved as `algorithm_used` for every run. `cs422cc` needed no special handling anywhere in the client.

3. Experiments

Setup

```
.venv/bin/python run_experiments.py listed_iperf3_servers.csv \
    -n 10 --duration 10 --interval 0.2 --repetitions 3 --seed 422 \
    --algorithms cs422cc cubic reno bbr
```

- **Destinations (10):** Dallas, Plano, Seattle, Salt Lake City, and San Jose (US); Toronto; Istanbul; Fuzhou (China); Singapore; Sydney. Three servers rejected the test during selection and were replaced automatically.
- **Duration** 10 s per test, **sampling interval** 0.2 s (50 samples per test), the same for all four algorithms.
- **Repetitions:** 3. **Test order:** for each repetition, for each destination, for each algorithm (cs422cc, cubic, reno, bbr). That is 120 tests, run between 00:40 and 01:09 UTC on Oct 23, 2026.
- **Every test succeeded,** and `algorithm_used` matched the requested algorithm in all 120 (`metadata.json`).
- Each sample logs cwnd, smoothed RTT, `min_rtt`, retransmitted segments in the interval (the loss signal), and goodput, all read from `TCP_INFO` on the data socket. Raw data: `results/20261023-004005/samples.csv`.

We ran the campaign on a VPS rather than on WSL2. A partial WSL2 campaign showed RTT jitter of several milliseconds on WSL2's virtual network. DGL's thresholds are fractions of `min_rtt` (about 3.4 ms for a 27 ms path), so that jitter alone tripped its slow-start exit and delay backoff: on one path cs422cc reached 30 Mbit/s against CUBIC's 617. The VPS's jitter is about 0.1 ms (ping `mdev`), which tests the algorithm rather than the virtual network.

Time series and scatter plots

The representative destination is `59.56.22.40:5201` **(Fuzhou, China, min RTT about 200 ms)**, the same for all four algorithms. It is the one destination where losses occurred for every algorithm, so it shows how each one reacts to loss. The runner's automatic pick, `148.230.59.38`, held all four algorithms at the same 440 Mbit/s with no loss, so its plots would show no differences. The plots were regenerated with `run_experiments.py --report-only results/20261023-004005 --representative 59.56.22.40:5201`.

Each figure shows all three repetitions; every algorithm's figures use the same axis limits (`compute_axis_limits()`). Time series are generated by `plot_time_series()` and scatter plots by `plot_scatter()`.

Aggregate statistics

Pooled over all destinations and repetitions, 1,500 samples per algorithm (`summarize_by_algorithm()`, `algorithm_summary.csv`). RTT is the kernel's smoothed RTT, so it includes any queueing an algorithm caused.

Algorithm	Goodput min / median / mean / p95 (Mbit/s)	RTT min / median / mean / p95 (ms)
cs422cc	0.0 / 164.6 / 496.7 / 1837.2	11.2 / 48.8 / 96.7 / 243.3
cubic	0.0 / 440.1 / 543.9 / 1826.1	11.1 / 48.4 / 96.2 / 227.4
reno	0.0 / 440.5 / 591.5 / 1837.7	11.2 / 48.7 / 95.9 / 232.0
bbr	0.0 / 447.8 / 590.0 / 1840.0	11.2 / 48.3 / 94.8 / 230.7

The pooled numbers mix paths from 11 ms to 215 ms and from 110 to 1,900 Mbit/s, so they are best read with the per-destination medians (`goodput_summary.csv`, from `summarize_goodput()`):

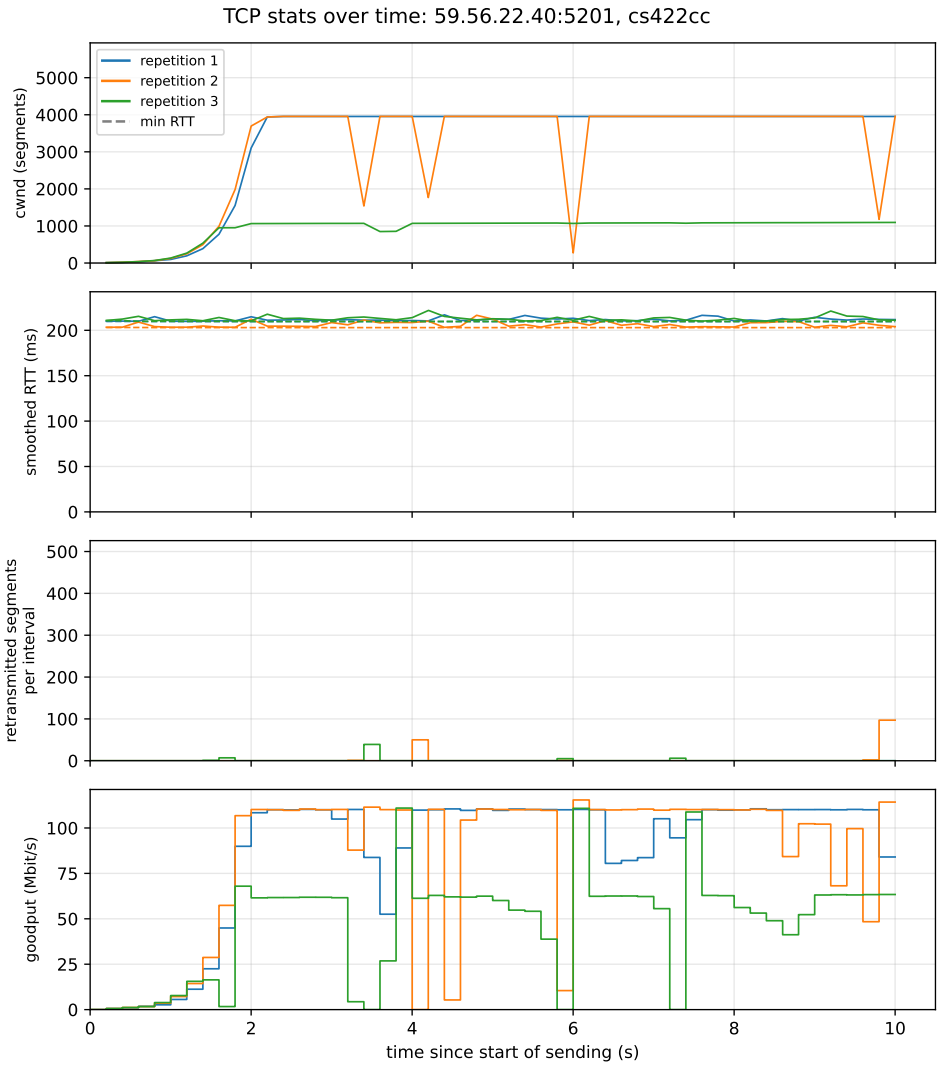


Figure 1: cs422cc time series, 59.56.22.40. cwnd dips at each loss and returns to its plateau within one sample.

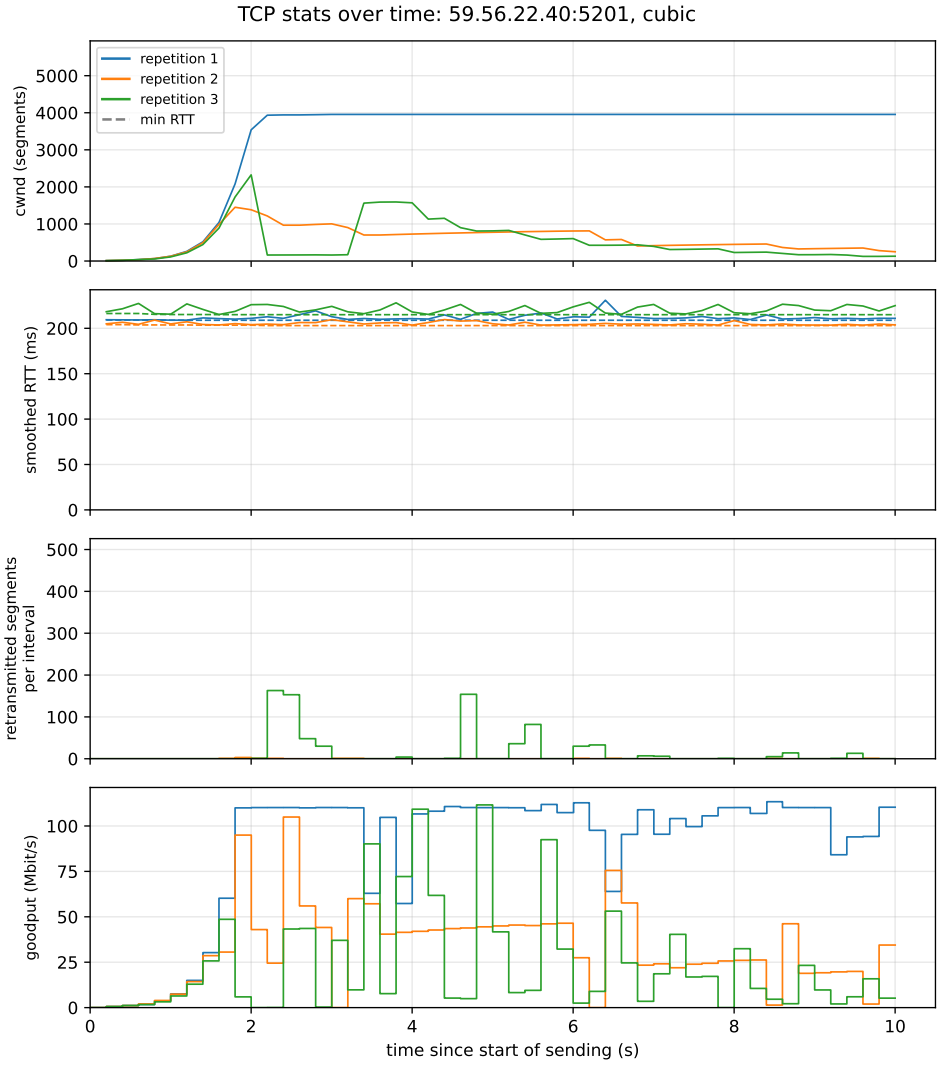


Figure 2: CUBIC time series, 59.56.22.40. After the first losses cwnd stays low for the rest of the test in repetitions 2 and 3.

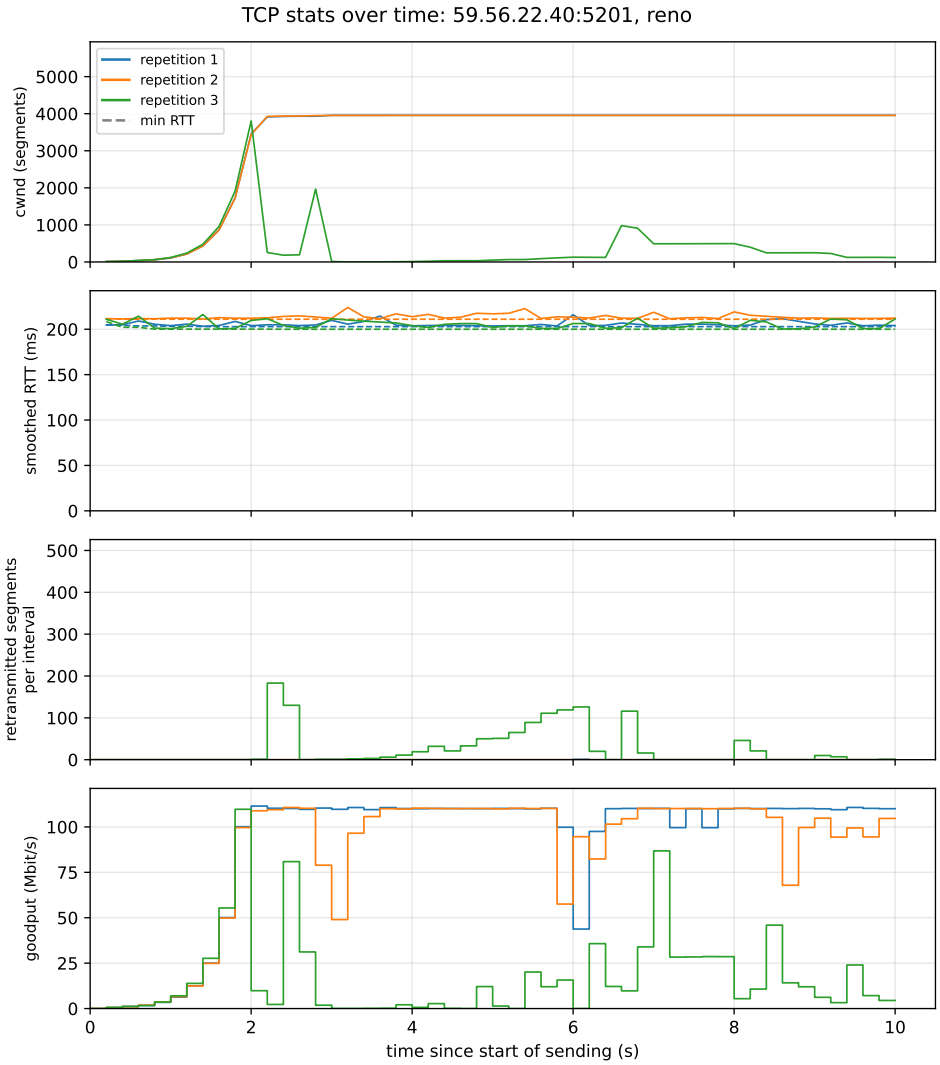


Figure 3: Reno time series, 59.56.22.40.

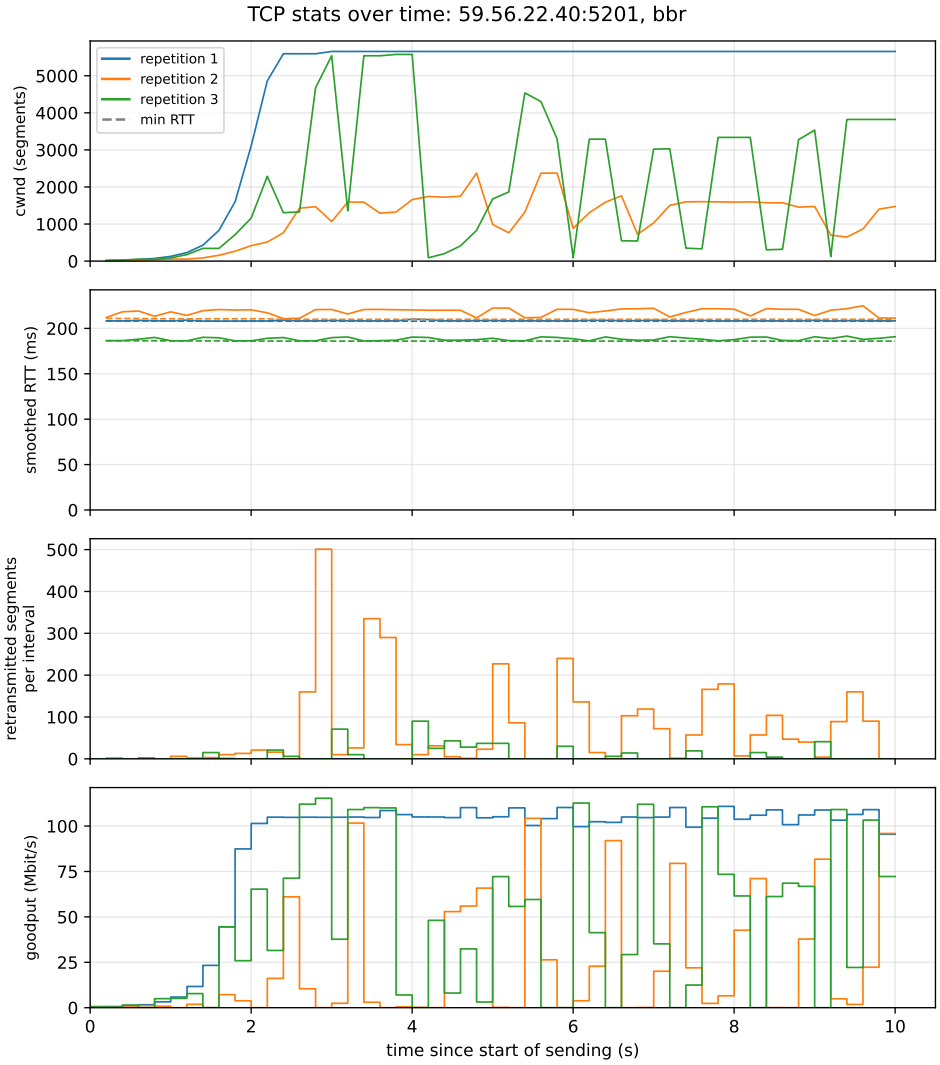


Figure 4: BBR time series, 59.56.22.40. Repetition 2 retransmits up to 500 segments per 0.2 s interval.

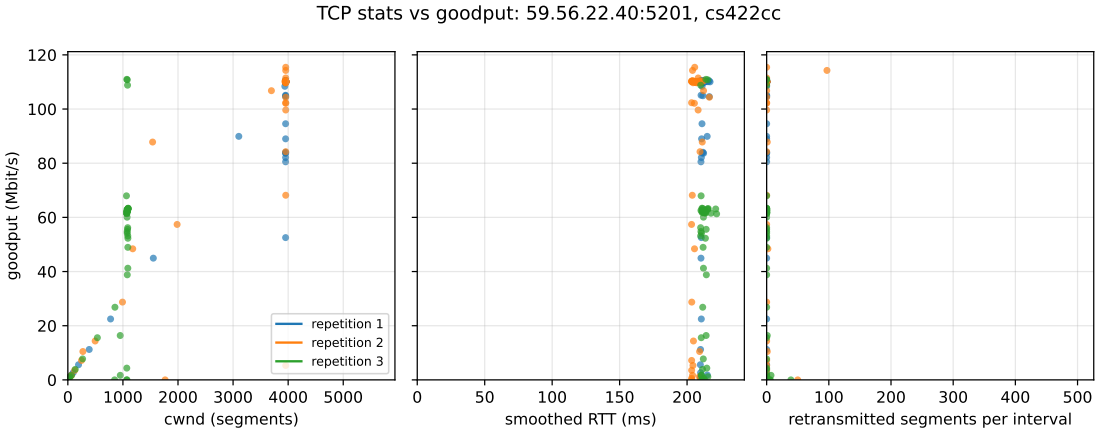


Figure 5: cs422cc scatter plots, 59.56.22.40.

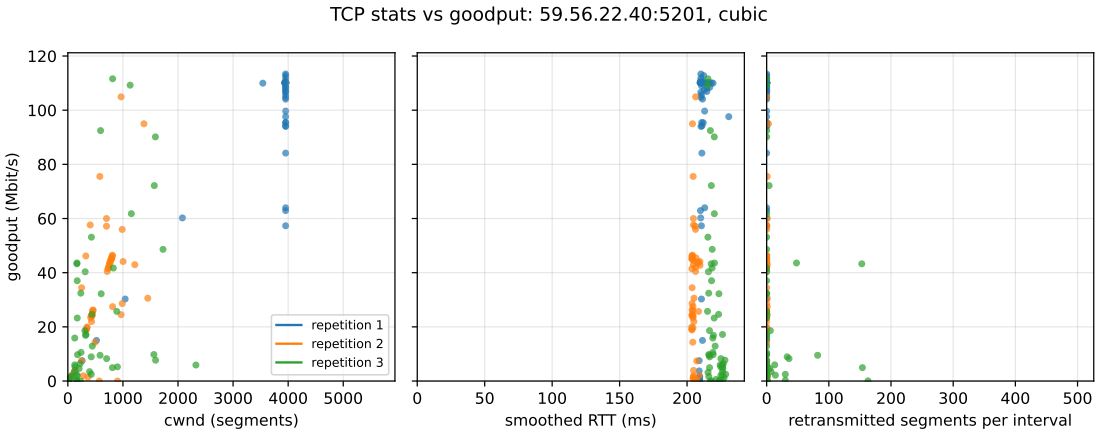


Figure 6: CUBIC scatter plots, 59.56.22.40.

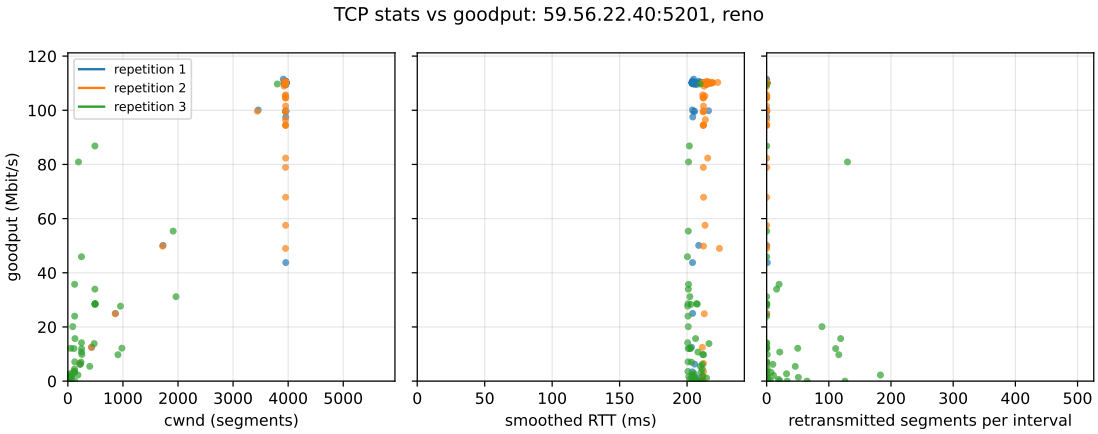


Figure 7: Reno scatter plots, 59.56.22.40.

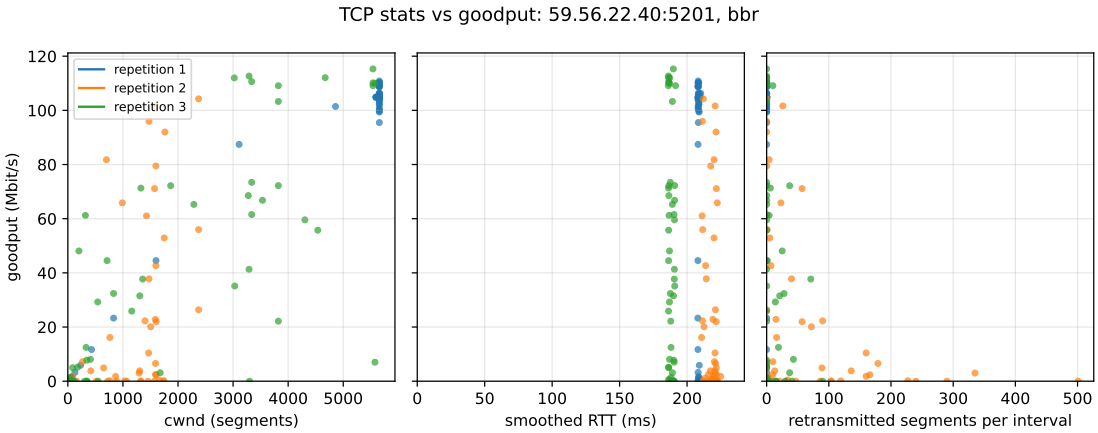


Figure 8: BBR scatter plots, 59.56.22.40.

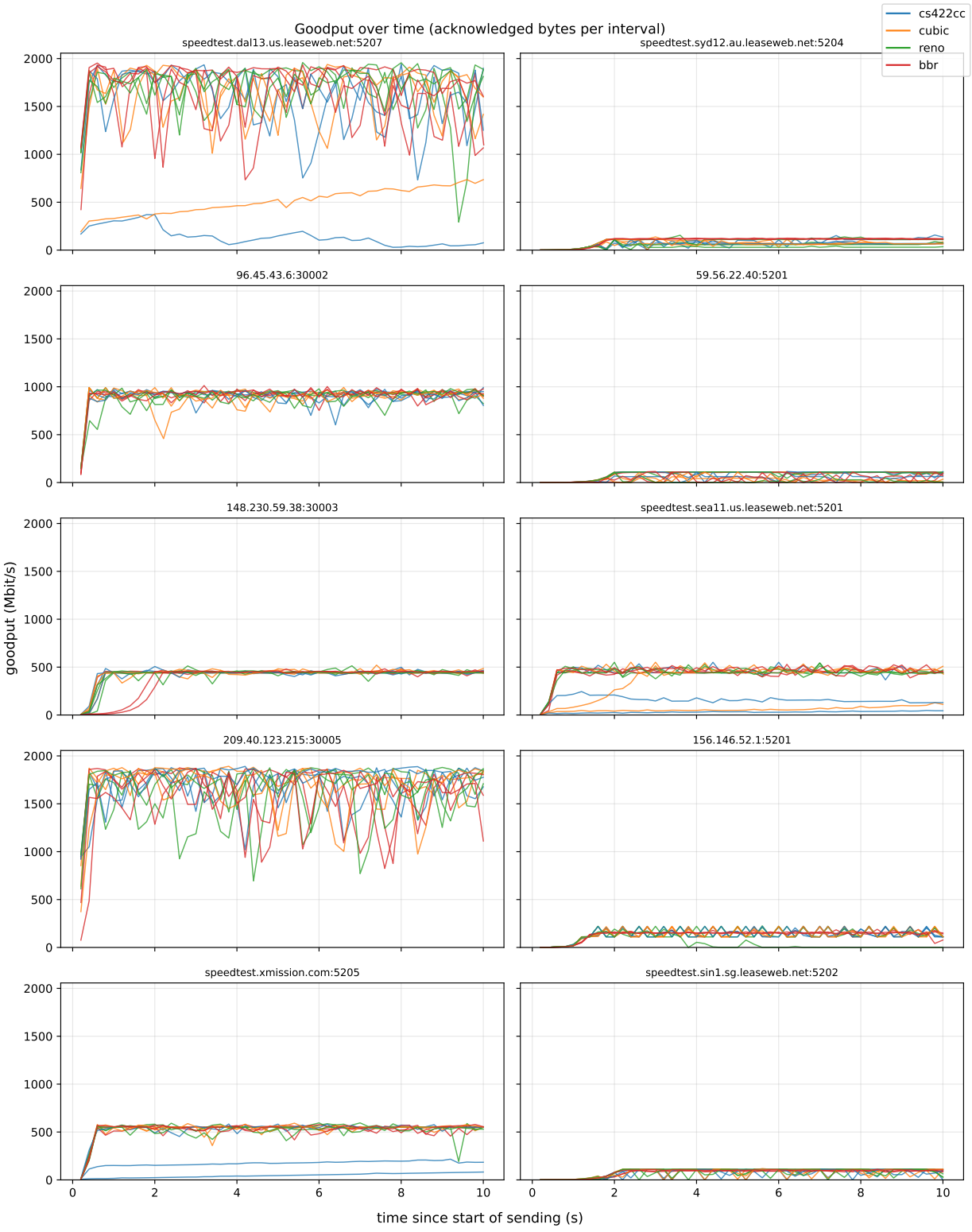


Figure 9: Goodput over time for all 10 destinations, one line per run, colored by algorithm. Generated by `plot_goodput_over_time()`.

Destination	min RTT	cs422cc	cubic	reno	bbr
Dallas (dal13)	11 ms	1453	1525	**1792**	1777
Plano (209.40.123.215)	11 ms	**1788**	1775	1720	1701
Toronto (96.45.43.6)	23 ms	925	916	922	**928**
Salt Lake (xmission)	39 ms	176	**550**	550	550
Seattle (sea11)	45 ms	151	440	441	**456**
San Jose (148.230.59.38)	48 ms	440	441	441	**448**
Istanbul (156.146.52.1)	142 ms	113	110	110	**151**
Sydney (syd12)	184 ms	67	63	57	**115**
Fuzhou (59.56.22.40)	186 ms	84	40	**95**	45
Singapore (sin1)	215 ms	110	110	**110**	94

Median goodput in Mbit/s; the highest in each row is bold.

Per run (each run's mean goodput compared with the best run on the same destination, any algorithm):

Algorithm	runs within 80% of best	runs below 50% of best	mean retransmits per run	runs with no retransmits
cs422cc	21 / 30	5 / 30	278	17 / 30
cubic	23 / 30	4 / 30	111	21 / 30
reno	26 / 30	3 / 30	194	18 / 30
bbr	28 / 30	2 / 30	162	21 / 30

Observed differences

1. A send-buffer limit sets the ceiling, for every algorithm. On 9 of the 10 destinations, the best goodput times the path's minimum RTT comes to 2.6 to 3.0 MB in flight, whether the RTT is 11 ms (1,911 Mbit/s) or 215 ms (110 Mbit/s). That is the sender's socket buffer: the host's `net.ipv4.tcp_wmem` maximum is 4 MiB, of which roughly 2.7 MB can carry payload. Once that much is in flight the connection is no longer limited by cwnd, so cwnd stops growing: the flat cwnd plateaus near 3,950 segments for cs422cc, CUBIC, and Reno (about 5,800 for BBR) in the time series are this limit, not the network. Our Assignment 2 campaign found the same ~2.8 MB limit. As a result, the four algorithms differ only in **how quickly they reach the ceiling** and **how they behave after a loss**, which is where the results below come from.

2. Startup: DGL sometimes leaves slow start too early. The delay gate ends slow start once `srtt - min_rtt > min_rtt / 8`. A doubling window sends bursts that briefly queue at the bottleneck, and on some runs that crossed the threshold almost immediately: in 4 of 30 runs, cs422cc left slow start within 0.6 s, with no retransmissions and cwnd between 41 and 487 segments (CUBIC's HyStart did the same in 2 runs, both on Seattle). After that, DGL grows by one segment per RTT, which on a 40 ms path adds only about 250 segments in 10 s, against a ceiling of about 2,000. This is why cs422cc is far behind on Salt Lake (176 vs 550 Mbit/s) and Seattle (151 vs 440), and why its repetitions vary so much there: Salt Lake gave 46, 177, and 550 Mbit/s medians, and the 550 run is the one where slow start continued to the ceiling. When slow start does not exit early, cs422cc matches the others (San Jose, Toronto, Plano, Singapore).

3. Loss without a queue: DGL holds its rate where the others collapse. Fuzhou is the regime the design targeted: a long path with losses but almost no queueing (median smoothed RTT within 1 to 2% of `min_rtt` for every algorithm). Mean goodput per repetition:

Fuzhou	rep 1	rep 2	rep 3
cs422cc	86.7	82.8	47.3
cubic	87.1	32.4	23.7
reno	89.9	85.5	16.8

Fuzhou	rep 1	rep 2	rep 3
bbr	87.6	22.5	44.6

In the time series, each cs422cc loss shows as a sharp dip in cwnd that recovers to the plateau within one 0.2 s sample (repetition 2, around 3.4, 4.2, 6, and 9.8 s). With no queue, DGL classifies these as random and returns `cwnd * 7/8`, which is still above what the send buffer lets it use, so goodput barely changes. CUBIC (multiplying by 0.7) and Reno (halving) treat every loss as congestion; at 200 ms RTT their regrowth is slow, so after a few losses cwnd stays at a few hundred segments for the rest of the test. BBR does not cut on loss at all, but in repetition 2 it kept sending into the loss, retransmitting 3,497 segments (up to 500 per interval) with its median RTT 10 ms above `min_rtt`, and averaged 22.5 Mbit/s. In cs422cc's third repetition, a loss during slow start left cwnd near 1,060, and at one segment per 200 ms round trip it only grows by about 40 segments in the remaining 8 s: the slow additive increase at long RTT that CUBIC was designed to fix.

4. RTT barely differs. Every algorithm's median smoothed RTT is within 1 to 2% of the path's `min_rtt` (median queueing delay: bbr 0.51 ms, cubic 0.63, cs422cc 0.70, reno 0.79). A sender limited to about 2.7 MB in flight does not fill a bottleneck queue on these paths, so DGL's delay backoff rarely triggered, and its RTT advantage from the design (keeping the queue below `min_rtt / 4`) had no queue to act on. The pooled RTT statistics are nearly identical across algorithms because they are dominated by which paths were measured. cs422cc's higher pooled p95 RTT (243 vs 227 to 232 ms) is an artifact of that mixing, not more queueing: on the four long-RTT destinations alone, cs422cc's p95 is 247.4 ms and CUBIC's 247.3 ms.

5. More retransmissions, by design. cs422cc has the highest mean retransmissions per run (278), though 17 of its 30 runs had none. Treating loss without a queue as random means continuing to send into it, where CUBIC and Reno back off. On Fuzhou that trade paid off; on a path where the loss is a real capacity limit without a visible queue, it would be unfriendly to other flows, which is what the 5% loss-rate guard is meant to catch.

Variability and limitations

- **Run-to-run variability is large** on the public Internet. The same algorithm on the same destination differed by up to 12x across repetitions (cs422cc on Salt Lake, 46 vs 550 Mbit/s), and losses on Fuzhou appeared in some repetitions and not others. Three repetitions are enough to show the regimes, not to rank close results: on most destinations the four medians are within a few percent.
- **10-second tests weight startup heavily.** A slow-start exit at 0.2 s costs most of a 10 s test, and the time to reach the send-buffer ceiling is a large share of the total.
- **The send buffer capped most tests.** With the 4 MiB `tcp_wmem` limit, no algorithm could use more than about 2.7 MB in flight, which hides differences in how far each would have grown. Raising `tcp_wmem` would separate them further.
- **Paths change between tests.** The four algorithms on one destination run about 15 s apart, so cross traffic and routes can differ between them.
- **Pooled statistics mix very different paths,** so the per-destination table is the fairer comparison.
- **The measurement host differs from Assignment 2's** (a VPS on kernel 6.8 instead of WSL2 on kernel 6.18), because WSL2's RTT jitter interferes with a delay-based algorithm. This is itself a limitation of DGL: its thresholds are relative to `min_rtt`, so absolute jitter of a few ms breaks them on short paths, as the Assignment 2 design assumed it would not.

Reproducing the results

On an Ubuntu 22.04+ Linux host (not a container: the program is loaded into the host kernel), with BTF, from the repository root:

```
sudo apt install -y build-essential clang llvm libbpf-dev python3-venv \
    linux-tools-common linux-tools-$(uname -r)
cc/load.sh
```

```
python3 -m venv .venv && .venv/bin/pip install -r requirements.txt
.venv/bin/python run_experiments.py listed_iperf3_servers.csv \
    -n 10 --duration 10 --interval 0.2 --repetitions 3 --seed 422 \
    --algorithms cs422cc cubic reno bbr
```

Results (`samples.csv`, `metadata.json`, `goodput_summary.csv`, `algorithm_summary.csv`, and the PDF plots) go to `results/<timestamp>/`. `--seed 422` reproduces the order in which destinations are tried; which servers are accepted, and every measurement, depend on the Internet at the time. To rebuild the summaries and plots from saved data:

```
.venv/bin/python run_experiments.py --report-only results/20261023-004005 \
    --representative 59.56.22.40:5201
```

The report is built from the repository root with `pandoc report/report.md -o report/report.pdf` (needs pandoc and a LaTeX distribution).

Use of AI tools

OpenAI Codex and Anthropic Claude were used to generate complete sections of code, stress-test the implementation, run the measurement campaign, and draft and polish this report. All AI-generated work was reviewed and verified by the group.